

NORMALIZAÇÃO

de Banco de Dados – Biblioteca Universitária

Etapa 3 — Análise Formal das Formas Normais

RESERVA

MULTA

TOMBAMENTO

Tópicos

01

Conceitos Fundamentais de Normalização

02

Tabela RESERVA — Análise 1FN, 2FN, 3FN, BCNF

03

Tabela MULTA — Análise 1FN, 2FN, 3FN, BCNF

04

Tabela EXEMPLAR — Análise 1FN, 2FN, 3FN, BCNF

05

Conclusão & Trade-offs

Conceitos Fundamentais

1FN

Primeira Forma Normal

Atributos atômicos, sem grupos repetitivos e sem atributos multivalorados. Cada célula contém um único valor indivisível.

2FN

Segunda Forma Normal

Estar em 1FN + nenhum atributo não-chave possui dependência parcial em relação a qualquer chave candidata.

3FN

Terceira Forma Normal

Estar em 2FN + sem dependências funcionais transitivas (não-chave $\rightarrow$ não-chave).

BCNF

Forma Normal de Boyce-Codd

Para toda DF $X \rightarrow Y$, X deve ser superchave. Mais restrita que 3FN.

 Normalização = eliminar redundância + garantir integridade, não apenas decompor tabelas.

Tabela RESERVA — Estrutura & Dependências Funcionais

Tabela Inicial (não normalizada)

ID_Reserva | Data_Reserva | Data_Validade | Status | ID_Usuario | Nome_Usuario | ID_Tombo | Nome_Exemplar

⚠ Problema: Nome_Usuario depende de ID_Usuario, e Nome_Exemplar depende de ID_Tombo — dependências transitivas (ID_Reserva → ID_Usuario → Nome_Usuario)!

Dependências Funcionais Identificadas:

ID_Reserva → Data_Reserva, Data_Validade, Status, ID_Usuario, ID_Tombo

[DF direta (PK)]

ID_Usuario → Nome_Usuario

[DF transitiva]

ID_Tombo → Nome_Exemplar

[DF transitiva]

Tabela RESERVA — Análise das Formas Normais

1FN

Satisfaz a 1FN ✓

Todos os atributos são atômicos e indivisíveis. Não há grupos repetitivos nem atributos multivalorados. Existe uma chave primária simples (ID_Reserva).

2FN

Satisfaz a 2FN ✓

A PK é simples (ID_Reserva). Com chave simples não é possível existir dependência parcial. Todos os atributos dependem totalmente de ID_Reserva.

3FN

Não satisfaz a 3FN ✗

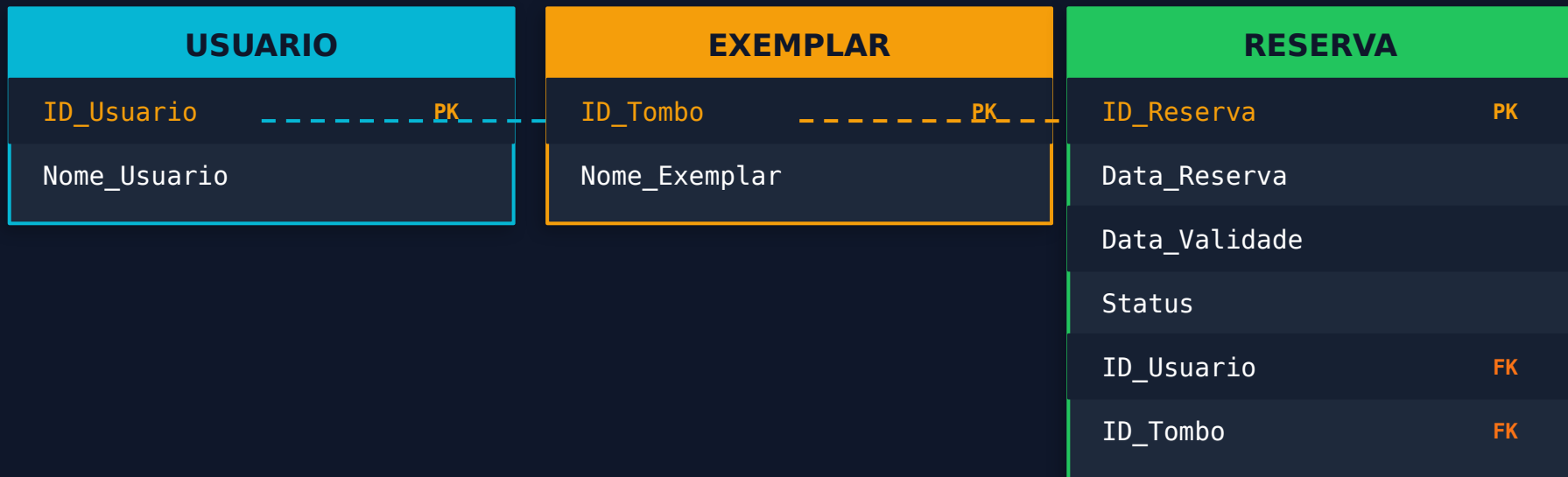
Existem dependências transitivas: ID_Reserva → ID_Usuario → Nome_Usuario e ID_Reserva → ID_Tombo → Nome_Exemplar. Atributos não-chave determinam outros atributos não-chave.

BCNF

Não satisfaz a BCNF ✗

As DFs ID_Usuario → Nome_Usuario e ID_Tombo → Nome_Exemplar têm determinantes que não são superchaves de RESERVA. Após decomposição em 3 tabelas, 3FN e BCNF são satisfeitas.

Tabela RESERVA — Esquema Normalizado Resultante



Cada tabela contém apenas atributos que dependem diretamente da chave primária, eliminando as dependências transitivas.

Tabela MULTA — Estrutura & Dependências Funcionais

MULTA

id_multa	INTEGER	PK
valor	DECIMAL(10,2)	
data_geracao	DATE	
data_pagamento	DATE	
dias_atraso	INTEGER	
status	VARCHAR(20)	
id_emprestimo	INTEGER	FK
id_motivo	INTEGER	FK

Dependências Funcionais

`id_multa` → `valor`, `data_geracao`,
`data_pagamento`, `dias_atraso`,
`status`, `id_emprestimo`, `id_motivo`

Obs.: id_emprestimo e id_motivo são chaves estrangeiras. Os atributos dessas entidades residem em suas próprias tabelas normalizadas.

Tabela MULTA — Análise das Formas Normais

1FN

Satisfaz a 1FN ✓

Todos os atributos são atômicos (valores indivisíveis). Não há grupos repetitivos nem atributos multivalorados. Existe uma chave primária simples (id_multa).

2FN

Satisfaz a 2FN ✓

A chave primária é simples (id_multa), portanto não há como existir dependência parcial. Todos os atributos dependem totalmente de id_multa.

3FN

Satisfaz a 3FN ✓

Não existem atributos não-chave dependendo de outros atributos não-chave dentro da própria relação MULTA. id_emprestimo e id_motivo são FKs cujos atributos residem em tabelas próprias.

BCNF

Satisfaz a BCNF ✓

Para toda DF identificada, o lado esquerdo é superchave (id_multa). Não há DF onde a determinante não seja superchave. A tabela está em BCNF.

Tabela EXEMPLAR — Estrutura & Dependências Funcionais

EXEMPLAR		
id_tombo	VARCHAR(20)	PK
tipo_aquisicao	VARCHAR(30)	
data_aquisicao	DATE	
valor_compra	DECIMAL(10,2)	
status	VARCHAR(20)	
conservacao	VARCHAR(30)	
id_localizacao	VARCHAR(50)	FK

LOCALIZACAO_FISICA		
id_localizacao	VARCHAR(50)	PK
endereco	VARCHAR(50)	
andar	INTEGER	
estante	VARCHAR(30)	

DFs

DF1: id_tombo → tipo_aquisicao, data_aquisicao, valor_compra, status, conservacao, id_localizacao

DF2: id_localizacao → endereco, andar, estante [dependência transitiva]

Tabela EXEMPLAR — Análise das Formas Normais

1FN

Satisfaz a 1FN ✓

Todos os atributos são atômicos e indivisíveis. Não há grupos repetitivos nem atributos multi-valorados. PK simples definida (id_tombo).

2FN

Satisfaz a 2FN ✓

A PK é simples (id_tombo), portanto não há como existir dependência parcial. Todos os atributos de EXEMPLAR dependem totalmente de id_tombo.

3FN

Não satisfaz a 3FN ✗

Existe dependência transitiva: id_tombo → id_localizacao → {endereco, andar, estante}. Atributos de localização dependem de id_localizacao (não-chave), não de id_tombo diretamente.

BCNF

Não satisfaz a BCNF ✗

A DF id_localizacao → {endereco, andar, estante} tem como determinante id_localizacao, que NÃO é superchave de EXEMPLAR. Viola a BCNF. Solução: separar LOCALIZACAO_FISICA em tabela própria (já feita no esquema).

Decomposição em EXEMPLAR + LOCALIZACAO_FISICA elimina a transitividade e satisfaz 3FN e BCNF.

Resumo Comparativo — Situação das Formas Normais

Tabela	1FN	2FN	3FN	BCNF
MULTA	✓ SIM	✓ SIM	✓ SIM	✓ SIM
RESERVA (inicial)	✓ SIM	✓ SIM	✗ NÃO	✗ NÃO
RESERVA (normalizada)	✓ SIM	✓ SIM	✓ SIM	✓ SIM
EXEMPLAR (inicial)	✓ SIM	✓ SIM	✗ NÃO	✗ NÃO
EXEMPLAR + LOCALIZAÇÃO	✓ SIM	✓ SIM	✓ SIM	✓ SIM

- RESERVA inicial falha na 3FN e BCNF por conter dependências transitivas (não dependências parciais — a PK é simples).
- EXEMPLAR inicial viola 3FN/BCNF pela dependência transitiva via `id_localizacao`. Após decomposições, todas as FNs são satisfeitas.

Conclusão & Trade-offs de Normalização



Redundância eliminada

As decomposições garantem que cada fato seja armazenado uma única vez, evitando anomalias de inserção, atualização e deleção.



Trade-off: Performance vs Integridade

BCNF pode exigir mais JOINS. Em casos de alta carga de leitura, pode-se aceitar desnormalização controlada com justificativa formal.



Integridade referencial mantida

FKs (id_emprestimo, id_usuario, id_tombo, id_localizacao) garantem consistência entre tabelas após as decomposições.



Normalização $\neq$ apenas decompor

O objetivo central é compreender as dependências funcionais e tomar decisões baseadas na semântica dos dados.

-- 1. DDL - CRIAÇÃO DAS TABELAS (ORDEM DE DEPENDÊNCIA)

-- 1.1 MÓDULO: USUÁRIO E ENDEREÇO

```
CREATE TABLE ENDERECO (  
    id_endereco SERIAL PRIMARY KEY,  
    cep CHAR(9) NOT NULL,  
    logradouro VARCHAR(120) NOT NULL,  
    numero VARCHAR(10) NOT NULL,  
    bairro VARCHAR(60) NOT NULL,  
    cidade VARCHAR(60) NOT NULL,  
    CONSTRAINT ck_cep_formato CHECK (cep ~ '^[0-9]{5}-[0-9]{3}$')  
);
```

```
CREATE TABLE USUARIO (  
    id_usuario SERIAL PRIMARY KEY,  
    primeiro_nome VARCHAR(50) NOT NULL,  
    sobrenome VARCHAR(80) NOT NULL,  
    cpf CHAR(14) NOT NULL UNIQUE,  
    email VARCHAR(120) NOT NULL UNIQUE,  
    data_nascimento DATE NOT NULL,  
    data_cadastro DATE NOT NULL DEFAULT CURRENT_DATE,  
    tipo_usuario VARCHAR(20) NOT NULL,  
    id_endereco INTEGER NOT NULL,  
    CONSTRAINT fk_usuario_endereco FOREIGN KEY (id_endereco) REFERENCES ENDERECO(id_endereco) ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT ck_usuario_tipo CHECK (tipo_usuario IN ('ALUNO', 'PROFESSOR', 'FUNCIONARIO')),  
    CONSTRAINT ck_data_nascimento CHECK (data_nascimento < CURRENT_DATE)  
);
```

```
CREATE TABLE TELEFONE_USUARIO (  
    id_usuario INTEGER NOT NULL,  
    telefone VARCHAR(20) NOT NULL,  
    CONSTRAINT pk_telefone_usuario PRIMARY KEY (id_usuario, telefone),  
    CONSTRAINT fk_telefone_usuario FOREIGN KEY (id_usuario) REFERENCES USUARIO(id_usuario) ON DELETE CASCADE ON UPDATE CASCADE  
);
```

-- 1.2 MÓDULO: ACERVO

```
CREATE TABLE ACERVO (  
    id_acervo INTEGER NOT NULL,  
    titulo VARCHAR(150) NOT NULL,  
    subtítulo VARCHAR(150),  
    ano_publicacao INTEGER,  
    idioma VARCHAR(50),  
    descricao TEXT,  
    CONSTRAINT pk_acervo PRIMARY KEY (id_acervo),  
    CONSTRAINT ck_acervo_ano CHECK (ano_publicacao > 0)  
);
```

```
CREATE TABLE LIVRO (  
    id_acervo INTEGER NOT NULL,  
    isbn VARCHAR(20) NOT NULL,  
    numero_paginas INTEGER,  
    edicao INTEGER,  
    CONSTRAINT pk_livro PRIMARY KEY (id_acervo),  
    CONSTRAINT fk_livro_acervo FOREIGN KEY (id_acervo) REFERENCES ACERVO (id_acervo) ON DELETE CASCADE ON UPDATE CASCADE,  
    CONSTRAINT uq_livro_isbn UNIQUE (isbn),  
    CONSTRAINT ck_livro_paginas CHECK (numero_paginas > 0)  
);
```

```
CREATE TABLE PERIODICO (  
    id_acervo INTEGER NOT NULL,  
    issn VARCHAR(10) NOT NULL,  
    volume INTEGER,  
    numero_edicao INTEGER,  
    periodicidade VARCHAR(50),  
    CONSTRAINT pk_periodico PRIMARY KEY (id_acervo),  
    CONSTRAINT fk_periodico_acervo FOREIGN KEY (id_acervo) REFERENCES ACERVO (id_acervo) ON DELETE CASCADE ON UPDATE CASCADE,  
    CONSTRAINT uq_periodico_issn UNIQUE (issn),  
);
```

```

        CONSTRAINT ck_periodico_volume CHECK (volume > 0),
        CONSTRAINT ck_periodico_numero CHECK (numero_edicao > 0)
    );

CREATE TABLE EBOOK (
    id_acervo INTEGER NOT NULL,
    formato_arquivo VARCHAR(20),
    tamanho_arquivo_mb DECIMAL(6,2),
    url_acesso TEXT,
    licenca_digital VARCHAR(50),
    CONSTRAINT pk_ebook PRIMARY KEY (id_acervo),
    CONSTRAINT fk_ebook_acervo FOREIGN KEY (id_acervo) REFERENCES ACERVO (id_acervo) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT ck_ebook_tamanho CHECK (tamanho_arquivo_mb > 0)
);

-- 1.3 MÓDULO: TOMBAMENTO
CREATE TABLE LOCALIZACAO_FISICA (
    id_localizacao VARCHAR(50) PRIMARY KEY,
    endereco VARCHAR(50) NOT NULL,
    andar INTEGER CHECK (andar >= 0),
    estante VARCHAR(30)
);

CREATE TABLE EXEMPLAR (
    id_tombo VARCHAR(20) PRIMARY KEY,
    tipo_aquisicao VARCHAR(30),
    data_aquisicao DATE,
    valor_compra DECIMAL(10,2) DEFAULT 0.00 CHECK (valor_compra >= 0),
    status VARCHAR(20) DEFAULT 'DISPONIVEL' CHECK (status IN ('DISPONIVEL', 'EMPRESTADO', 'DANIFICADO', 'RESERVADO')),
    conservacao VARCHAR(30),
    id_localizacao VARCHAR(50) NOT NULL,
    id_acervo INTEGER NOT NULL,
    CONSTRAINT fk_exemplar_localizacao FOREIGN KEY (id_localizacao) REFERENCES LOCALIZACAO_FISICA(id_localizacao) ON DELETE CASCADE,
    CONSTRAINT fk_exemplar_acervo FOREIGN KEY (id_acervo) REFERENCES ACERVO(id_acervo) ON DELETE CASCADE
);

-- 1.4 MÓDULO: EMPRÉSTIMO, RESERVA E MULTA (Sintetizado)
CREATE TABLE EMPRESTIMO (
    id_emprestimo SERIAL PRIMARY KEY,
    id_usuario INTEGER NOT NULL,
    id_tombo VARCHAR(20) NOT NULL,
    status VARCHAR(20) NOT NULL,
    data_inicio DATE NOT NULL,
    data_retorno DATE,
    data_fim_previsto DATE NOT NULL,
    CONSTRAINT fk_emp_usuario FOREIGN KEY (id_usuario) REFERENCES USUARIO(id_usuario) ON DELETE RESTRICT,
    CONSTRAINT fk_emp_exemplar FOREIGN KEY (id_tombo) REFERENCES EXEMPLAR(id_tombo) ON DELETE RESTRICT,
    CONSTRAINT chk_status_emp CHECK (status IN ('ativo', 'devolvido', 'atrasado')),
    CONSTRAINT chk_data_retorno CHECK (data_retorno IS NULL OR data_retorno >= data_inicio)
);

CREATE TABLE RESERVA (
    id_reserva SERIAL PRIMARY KEY,
    id_usuario INTEGER NOT NULL,
    id_tombo VARCHAR(20) NOT NULL,
    data_reserva DATE NOT NULL,
    data_validade DATE NOT NULL,
    status VARCHAR(20) NOT NULL,
    CONSTRAINT fk_res_usuario FOREIGN KEY (id_usuario) REFERENCES USUARIO(id_usuario),
    CONSTRAINT fk_res_exemplar FOREIGN KEY (id_tombo) REFERENCES EXEMPLAR(id_tombo),
    CONSTRAINT chk_status_res CHECK (status IN ('ATIVA', 'FINALIZADA', 'CANCELADA')),
    CONSTRAINT chk_data_val CHECK (data_validade >= data_reserva)
);

CREATE TABLE MOTIVO_MULTA (
    id_motivo INTEGER PRIMARY KEY,
    descricao VARCHAR(255) NOT NULL
);

```

```

);

CREATE TABLE MULTA (
    id_multa SERIAL PRIMARY KEY,
    id_emprestimo INTEGER NOT NULL,
    id_motivo INTEGER NOT NULL,
    valor DECIMAL(10,2) NOT NULL,
    data_geracao DATE NOT NULL,
    data_pagamento DATE,
    dias_atraso INTEGER,
    status VARCHAR(20) NOT NULL,
    CONSTRAINT fk_multa_emp FOREIGN KEY (id_emprestimo) REFERENCES EMPRESTIMO(id_emprestimo),
    CONSTRAINT fk_multa_motivo FOREIGN KEY (id_motivo) REFERENCES MOTIVO_MULTA(id_motivo)
);

-- 2. DML - INSERÇÃO DE DADOS DE TESTE

-- Endereços
INSERT INTO ENDERECO (cep, logradouro, numero, bairro, cidade) VALUES
('30140-010', 'Rua da Bahia', '1000', 'Centro', 'Belo Horizonte'),
('30535-000', 'Av. Dom José Gaspar', '500', 'Coração Eucarístico', 'Belo Horizonte'),
('30120-000', 'Av. Afonso Pena', '150', 'Centro', 'Belo Horizonte'),
('31270-901', 'Av. Antônio Carlos', '6627', 'Pampulha', 'Belo Horizonte'),
('30310-000', 'Rua Grão Mogol', '200', 'Sion', 'Belo Horizonte'),
('32000-000', 'Praça da Matriz', '10', 'Centro', 'Contagem'),
('34000-000', 'Rua Direita', '50', 'Centro', 'Nova Lima'),
('30640-000', 'Av. Olineto Meireles', '300', 'Barreiro', 'Belo Horizonte'),
('30150-280', 'Rua Ceará', '450', 'Santa Efigênia', 'Belo Horizonte'),
('30411-000', 'Av. Amazonas', '2000', 'Prado', 'Belo Horizonte');

-- Usuários
INSERT INTO USUARIO (primeiro_nome, sobrenome, cpf, email, data_nascimento, tipo_usuario, id_endereco) VALUES
('João Otávio', 'Gurgel', '111.222.333-44', 'joao.gurgel@puc.br', '2006-04-25', 'ALUNO', 2),
('Cristiano', 'Macedo', '222.333.444-55', 'cristiano.macedo@puc.br', '1980-05-10', 'PROFESSOR', 1),
('Maria', 'Silva', '333.444.555-66', 'maria.silva@puc.br', '2000-01-15', 'ALUNO', 3),
('Ricardo', 'Oliveira', '444.555.666-77', 'ricardo.ol@puc.br', '1975-08-20', 'FUNCIONARIO', 4),
('Ana', 'Beatriz', '555.666.777-88', 'ana.b@puc.br', '2002-11-30', 'ALUNO', 5),
('Carlos', 'Eduardo', '666.777.888-99', 'carlos.ed@puc.br', '1995-03-22', 'ALUNO', 6),
('Fernanda', 'Lima', '777.888.999-00', 'fernanda.l@puc.br', '1988-12-05', 'PROFESSOR', 7),
('Lucas', 'Martins', '888.999.000-11', 'lucas.m@puc.br', '2001-07-14', 'ALUNO', 8),
('Juliana', 'Costa', '999.000.111-22', 'juliana.c@puc.br', '1992-06-18', 'FUNCIONARIO', 9),
('Amanda', 'Souza', '000.111.222-33', 'amanda.s@puc.br', '1998-09-09', 'ALUNO', 10);

-- Telefones
INSERT INTO TELEFONE_USUARIO (id_usuario, telefone) VALUES
(1, '(31) 98888-7777'), (1, '(31) 3333-2222'), (2, '(31) 97777-6666'), (3, '(31) 96666-5555'), (4, '(31) 3444-1111');

-- Acervo
INSERT INTO ACERVO VALUES
(1, 'Dom Casmurro', NULL, 1899, 'Português', 'Romance clássico brasileiro.'),
(2, 'Banco de Dados', 'Projeto e Implementação', 2020, 'Português', 'Livro acadêmico sobre bancos de dados.'),
(3, 'Engenharia de Software', NULL, 2019, 'Português', 'Livro sobre processos de software.'),
(4, 'Algoritmos', 'Teoria e Prática', 2018, 'Português', 'Livro sobre algoritmos e estruturas de dados.'),
(5, 'Revista Ciência Hoje', NULL, 2024, 'Português', 'Periódico científico nacional.'),
(6, 'Journal of Software Engineering', NULL, 2023, 'Inglês', 'Periódico internacional de engenharia de software.'),
(7, 'Revista Tecnologia e Sociedade', NULL, 2022, 'Português', 'Periódico sobre tecnologia.'),
(8, 'Computing Review', NULL, 2021, 'Inglês', 'Periódico da área de computação.'),
(9, 'Introdução ao PostgreSQL', NULL, 2023, 'Português', 'E-book sobre banco de dados PostgreSQL.'),
(10, 'Python para Iniciantes', NULL, 2022, 'Português', 'E-book introdutório de programação.'),
(11, 'Machine Learning Básico', NULL, 2024, 'Português', 'E-book sobre aprendizado de máquina.'),
(12, 'Data Science Handbook', NULL, 2021, 'Inglês', 'E-book sobre ciência de dados.');

-- Livros, Periódicos, Ebooks
INSERT INTO LIVRO VALUES (1, '978-85-333-0227-3', 320, 3), (2, '978-65-555-1234-8', 540, 1), (3, '978-85-7522-999-1', 480, 1),
INSERT INTO PERIODICO VALUES (5, '1234-5678', 12, 45, 'Mensal'), (6, '2345-6789', 8, 20, 'Trimestral'), (7, '3456-7890', 8, 12, 'Semestral'),
INSERT INTO EBOOK VALUES (9, 'PDF', 12.50, 'https://biblioteca.edu/ebooks/postgresql.pdf', 'Aberta'), (10, 'EPUB', 8.75, 'https://biblioteca.edu/ebooks/python.pdf', 'Aberta'),
-- Localização Física

```

```

INSERT INTO LOCALIZACAO_FISICA (id_localizacao, endereco, andar, estante) VALUES
('SALA101-A', 'Sala de Leitura - Ala A', 1, 'E1'), ('SALA101-B', 'Sala de Leitura - Ala B', 1, 'E3'), ('SALA202', 'Sala de Leitura - Ala C', 2, 'E2');

-- Exemplos
INSERT INTO EXEMPLAR (id_tombo, tipo_aquisicao, data_aquisicao, valor_compra, status, conservacao, id_localizacao, id_acervo) VALUES
('T2023001', 'Compra', '2023-05-10', 89.90, 'DISPONIVEL', 'Bom', 'SALA101-A', 1),
('T2023002', 'Doação', '2023-06-15', 0.00, 'EMPRESTADO', 'Ótimo', 'SALA101-B', 2),
('T2023003', 'Compra', '2023-08-20', 145.50, 'DISPONIVEL', 'Bom', 'SALA202', 3),
('T2023004', 'Compra', '2024-01-05', 67.80, 'DANIFICADO', 'Ruim', 'DEP03', 4);

-- Motivos de Multa
INSERT INTO MOTIVO_MULTA VALUES (1, 'Atraso na Devolução'), (2, 'Dano ao Material'), (3, 'Perda do Exemplar');

-- Empréstimos
INSERT INTO EMPRESTIMO (id_usuario, id_tombo, status, data_inicio, data_retorno, data_fim_previsto) VALUES
(1, 'T2023002', 'ativo', '2025-05-01', NULL, '2025-05-15'),
(2, 'T2023003', 'devolvido', '2025-04-10', '2025-04-20', '2025-04-25');

-- Reservas
INSERT INTO RESERVA (id_usuario, id_tombo, data_reserva, data_validade, status) VALUES
(3, 'T2023001', '2026-05-01', '2026-05-05', 'ATIVA');

-- Multas
INSERT INTO MULTA (id_emprestimo, id_motivo, valor, data_geracao, status) VALUES
(2, 1, 15.00, '2025-04-21', 'PENDENTE');

-- 3. CONSULTAS SQL (Q1 a Q10)

-- Q1: Projeção e seleção simples - Listar usuários do tipo 'ALUNO'
SELECT primeiro_nome, sobrenome, email
FROM USUARIO
WHERE tipo_usuario = 'ALUNO';

-- Q2: Seleção com operadores lógicos - Buscar acervos em português publicados após 2020
SELECT titulo, ano_publicacao, idioma
FROM ACERVO
WHERE idioma = 'Português' AND ano_publicacao > 2020;

-- Q3: INNER JOIN (2 tabelas) - Listar livros com seus respectivos ISBNs e títulos
SELECT A.titulo, L.isbn
FROM ACERVO A
INNER JOIN LIVRO L ON A.id_acervo = L.id_acervo;

-- Q4: INNER JOIN (3 tabelas) - Mostrar usuário, nome do exemplar e status de empréstimos ativos
SELECT U.primeiro_nome, A.titulo, E.status
FROM EMPRESTIMO E
JOIN USUARIO U ON E.id_usuario = U.id_usuario
JOIN EXEMPLAR EX ON E.id_tombo = EX.id_tombo
JOIN ACERVO A ON EX.id_acervo = A.id_acervo
WHERE E.status = 'ativo';

-- Q5: LEFT OUTER JOIN - Listar todos os endereços e os usuários vinculados (incluindo endereços vazios)
SELECT E.logradouro, U.primeiro_nome
FROM ENDereco E
LEFT JOIN USUARIO U ON E.id_endereco = U.id_endereco;

-- Q6: Agrupamento e agregação - Quantidade de acervos por idioma com mais de 2 itens
SELECT idioma, COUNT(*) AS quantidade
FROM ACERVO
GROUP BY idioma
HAVING COUNT(*) >= 2;

-- Q7: Subquery não correlacionada - Títulos de acervos que possuem exemplares com valor de compra acima da média
SELECT titulo
FROM ACERVO
WHERE id_acervo IN (
    SELECT id_acervo

```

```

FROM EXEMPLAR
WHERE valor_compra > (SELECT AVG(valor_compra) FROM EXEMPLAR)
);

-- Q8: EXISTS - Usuários que possuem pelo menos uma reserva ativa
SELECT U.primeiro_nome, U.email
FROM USUARIO U
WHERE EXISTS (
    SELECT 1 FROM RESERVA R
    WHERE R.id_usuario = U.id_usuario AND R.status = 'ATIVA'
);

-- Q9: VIEW - Criar visão do perfil completo do usuário (Nome, Cidade, Tipo)
CREATE OR REPLACE VIEW vw_perfil_usuario AS
SELECT u.id_usuario, u.primeiro_nome, e.cidade, u.tipo_usuario
FROM USUARIO u
JOIN ENDEREÇO e ON u.id_endereco = e.id_endereco;

SELECT * FROM vw_perfil_usuario;

-- Q10: Consulta de negócio livre - Ranking de materiais mais reservados por título
SELECT A.titulo, COUNT(R.id_reserva) AS total_reservas
FROM ACERVO A
JOIN EXEMPLAR EX ON A.id_acervo = EX.id_acervo
JOIN RESERVA R ON EX.id_tombo = R.id_tombo
GROUP BY A.titulo
ORDER BY total_reservas DESC;

```